DATA ENGINEER — COMPLETE JOB ROADMAP

Internship (Month 1-6) → AI Pipeline Specialist (Month 7-14) → 2029 Job Ready

Target: AI/ML Pipeline Engineering Specialization

TIMELINE OVERVIEW

```
Month 1-6   → Core DE Foundation (internship-ready)
              [Full detail in: INTERNSHIP_6MONTH_FINAL.md]

Month 7-9   → ML Pipeline Engineering
Month 10-11 → LLM Infrastructure
Month 12-13 → Real-Time AI Pipelines
Month 14    → Capstone: Production RAG Pipeline

Year 2-3    → On-the-job depth (Spark at scale, K8s, Terraform)
2029        → First job — AI/ML Data Engineer, 18-25 LPA target
```

PHASE 1: CORE DE FOUNDATION (Month 1-6)

Already covered in full detail in your internship roadmap. Summary: Python → SQL/PostgreSQL → pandas/ETL → Docker/Airflow → GCP/AWS → dbt/Spark/Kafka → DSA/Interview Prep → CP Capstone

This phase gets you internship-ready and lands you the first internship around Month 6-7.

PHASE 2: ML PIPELINE ENGINEERING (Month 7-9)

90 days | 4 hrs/day | 360 hrs

This is what separates a generic Data Engineer from an AI-ready one. You are not learning to build models. You are learning to build the infrastructure that feeds and monitors models.

Week 31-32 — MLflow + Experiment Tracking

Topic	Mini Project
What MLflow does (track experiments,	Log 5 different model runs with different

Topic	Mini Project
models, metrics)	parameters
MLflow Tracking Server setup	Run MLflow locally, view UI, compare runs
Model Registry	Register a model, move it through Staging → Production
Logging metrics, params, artifacts	Track accuracy/loss across 10 training runs of a simple model

🎯 **Checkpoint:** Train 3 versions of a simple classifier (different params), log all to MLflow, pick the best one based on tracked metrics.

Week 33-34 — Feature Stores

Topic	Mini Project
What a feature store is and why it exists	Write a 1-page doc: problem it solves (training/serving skew)
Feast — open source feature store	Install Feast, define a simple feature (user's avg order value)
Online vs Offline feature serving	Serve the same feature both ways, compare latency
Feature versioning	Update a feature definition, see how Feast handles versions

🎯 **Checkpoint:** Build a feature store for your Stock Price project — features like "7-day moving average," "volume spike flag," served both for training (offline) and prediction (online).

Week 35-36 — Data Quality for ML

Topic	Mini Project
Great Expectations — what it does	Define 10 expectations on your token analytics data
Data validation suites	Build a full validation suite, run it in a pipeline step
Data drift — what it means	Compare two time periods of data, detect distribution shift manually
Evidently AI	Generate a drift report comparing training vs production data

🎯 **Checkpoint:** Add a Great Expectations validation step + Evidently AI drift check to your Stock Price pipeline. Pipeline halts if drift detected.

Week 37-38 — ML Pipeline Orchestration

Topic	Mini Project
Training pipelines vs serving pipelines	Diagram both for a real use case
Airflow for ML — retrain DAGs	Build a weekly retrain DAG: extract → train → evaluate → register
Model evaluation gates	Add logic: only deploy if new model beats old model on test set
Model versioning + rollback	Simulate a bad model deploy, roll back to previous version

🎯 **PROJECT 7: "ML Retraining Pipeline"** Full Airflow DAG that retrains a model weekly, validates data quality, checks for drift, evaluates against current production model, only promotes if better, logs everything to MLflow.

Week 39 — scikit-learn for DE (not DS depth, just enough)

Topic	Mini Project
Train/test split, basic models	Train a simple regression and classifier
Model serialization (pickle, joblib)	Save and load a trained model
Basic evaluation metrics	Calculate accuracy, precision, recall, RMSE

PHASE 3: LLM INFRASTRUCTURE (Month 10-11)

60 days | 4 hrs/day | 240 hrs

Week 40-41 — LLM APIs + Cost Management

Topic	Mini Project
OpenAI, Anthropic, Gemini APIs	Call all 3 APIs with the same prompt, compare outputs

Topic	Mini Project
Token counting (tiktoken)	Build a token counter for any text before sending to API
Cost calculation per model	Build the PRICE_TABLE cost calculator (from your earlier project)
Prompt engineering basics	Write 5 prompt variations, compare quality vs cost

Week 42-43 — Vector Databases

Topic	Mini Project
What embeddings are	Generate embeddings for 20 sentences, visualize similarity
Pinecone or ChromaDB setup	Store and query embeddings in a vector DB
Similarity search	Build a "find similar documents" tool
Chunking strategies	Chunk a long PDF document 3 different ways, compare retrieval quality

🎯 **Checkpoint:** Build a simple semantic search tool over 50 documents using embeddings + vector DB.

Week 44-45 — RAG Pipelines

Topic	Mini Project
RAG architecture (retrieve → augment → generate)	Build a basic RAG pipeline over a small document set
LangChain basics	Rebuild the same RAG pipeline using LangChain
Context window management	Handle a case where retrieved docs exceed context limit
Evaluating RAG quality	Build a simple eval set: question → expected answer, measure accuracy

🎯 **PROJECT 8: "Company Docs Q&A Bot"** RAG pipeline that answers questions about a set of documents (your own notes/PDFs work fine). Ingestion → chunking → embeddings → vector DB → retrieval → LLM answer generation.

Week 46-47 — LLM Observability

Topic	Mini Project
LangSmith or Helicone	Add observability to your RAG pipeline — see every LLM call traced
Tracking latency, cost, quality per query	Build a dashboard: cost per query, latency distribution
Prompt versioning	Treat prompts like code — version them, track which version performs better
Hallucination detection basics	Build a simple fact-check step against your source documents

---

PHASE 4: REAL-TIME AI PIPELINES (Month 12-13)

60 days | 4 hrs/day | 240 hrs

Week 48-49 — Kafka Deep Dive (beyond basics)

Topic	Mini Project
Kafka Streams concepts	Build a stream processing job that aggregates events in windows
Schema Registry	Define an Avro schema for your events, enforce it
Exactly-once semantics	Understand and demonstrate idempotent producers

Week 50-51 — Spark Streaming

Topic	Mini Project
Structured Streaming basics	Read a live Kafka topic into Spark Streaming
Windowed aggregations	Calculate rolling 5-minute average of a streaming metric
Streaming + batch unification (Lambda arch)	Build both batch and streaming path for the same data, reconcile

Week 52-53 — Real-Time Feature Computation

Topic	Mini Project
Real-time features for ML serving	Compute "user activity in last 10 min" as a streaming feature
Connecting streaming pipeline to feature store	Push real-time features into Feast online store
Real-time model serving	Serve a model prediction using a real-time computed feature

🎯 **PROJECT 9: "Real-Time Token Cost Monitor"** Upgrade your AI Token Analytics project — instead of hourly batch, make it real-time using Kafka + Spark Streaming. Cost dashboard updates within seconds of an API call happening.

PHASE 5: CAPSTONE (Month 14)

30 days | 4 hrs/day | 120 hrs

🏆 **FINAL CAPSTONE: "Production RAG Pipeline with Full Observability"**

This combines everything from Phase 1-4 into one system.

```
DATA INGESTION
→ Scrape/collect documents daily (PDFs, web pages, whatever domain you
choose)
→ Store raw in GCS
→ Validate with pydantic + Great Expectations

VECTOR PIPELINE
→ Chunk documents (tested strategies from Week 43)
→ Generate embeddings
→ Store in Pinecone/ChromaDB
→ Incremental updates (only new/changed documents — not full reindex)

LLM LAYER
→ RAG system answers questions using vector store as context
→ Multiple LLM support (compare cost/quality across models)
→ Prompt versioning in place

OBSERVABILITY
→ Every LLM call logged (cost, latency, tokens) — your Token Analytics
  project plugged in here
→ Data drift monitoring on embeddings (Evidently AI)
→ Alert when retrieval quality drops
→ Full tracing with LangSmith/Helicone

ORCHESTRATION
→ Airflow DAG: daily ingestion + weekly reindex + weekly quality report
→ Data quality checks at every step
→ Retrain/re-embed triggers based on drift detection

SERVING
→ FastAPI endpoint serving the RAG system
→ Deployed on Cloud Run

DASHBOARD
→ BigQuery + Looker Studio
→ Query volume, cost per query, most common topics, system health,
  retrieval quality over time
```

This is the project that gets you 18-25 LPA, not 8-12 LPA. No other fresher in the market will have this combination of DE foundation + ML pipelines + LLM infra + real-time + observability.

WHAT YOU DO NOT NEED YET (Learn On The Job, Year 2-3)

```
☐ Kubernetes (deep)          → most companies use managed services
                               at fresher level you don't operate K8s
clusters
☐ Terraform / IaC            → learn when you join a team using it
☐ Multi-region architecture  → only senior engineers handle this
☐ Advanced Spark tuning      → comes with scale, not needed day one
☐ Data Mesh / Data Fabric    → organizational patterns, learn in context
☐ Full security/compliance   → GDPR depth, etc. — company will train you
```

Do not spend your limited time here now. Every hour spent on these is an hour not spent on what gets you hired by 2029.

COMPLETE SKILL MAP BY 2029

```
FOUNDATION (Month 1-6)
✅ Python, SQL/PostgreSQL, Linux
✅ pandas, APIs, ETL, Data Modeling
✅ Docker, Airflow
✅ GCP (deep), AWS (basics)
✅ dbt, Spark, Kafka basics
✅ DSA, SQL interviews, System Design

SPECIALIZATION (Month 7-14)
✅ MLflow, Feature Stores, Data Quality for ML
✅ LLM APIs, Vector DBs, RAG, LangChain
✅ LLM Observability (LangSmith/Helicone)
✅ Kafka Streams, Spark Streaming
✅ Real-time feature computation

PROJECTS BUILT
✅ 7 checkpoint projects (Month 1-6)
✅ ML Retraining Pipeline
✅ Company Docs Q&A Bot (RAG)
✅ Real-Time Token Cost Monitor
✅ CP Contest Intelligence Platform (capstone 1)
✅ Production RAG Pipeline (capstone 2 — final)
```

REALISTIC TIMELINE TO YOUR FIRST JOB

```
Month 6     → Internship (15-30k/month) — apply with CP Platform project
Month 14    → BTech still ongoing — keep building, intern again if possible
Year 3      → Final year — apply for full-time with BOTH capstones ready
2029        → First job — AI/ML Data Engineer
              Target: 18-25 LPA at a strong product company/startup
```

THE NON-NEGOTIABLE RULE

```
Depth over breadth. Always.

A fresher who deeply knows:
  SQL + Python + Airflow + 1 cloud + RAG pipelines + observability

beats a fresher who shallowly knows:
  Everything in the roadmap.sh diagram

Every single time. In every interview. At every company.
```

Month 1-6 gets you hired as an intern. Month 7-14 gets you hired at 18-25 LPA in 2029. Two capstones. One specialization. Zero wasted hours.